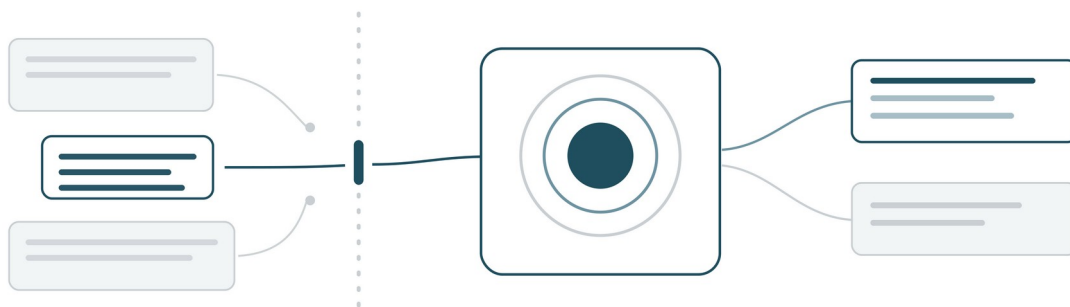




Donner une mémoire à un agent

Choisir ce qu'il retient, sur le cas d'un serveur MCP en local.

Denis Lamard

Concepteur et intégrateur d'agents IA

github.com/denislamard

Version 1.0, 2026

Sommaire

1. La mémoire qui se remplit toute seule
2. Ce que « choisir ce qui entre » veut dire
3. Neuf tools, deux familles
4. La consigne d'écriture n'est pas un verrou
5. Ce qu'un tool d'écriture rend
6. Le seul contrôle technique
7. Le trajet d'une note
8. Ce que le morceau emporte avec lui
9. Deux fois le même texte
10. Retrouver : hybride, puis reclassement
11. Ce que le score veut dire
12. Régler le seuil sur des cas de référence
13. Deux collections, une source de vérité
14. Ce qu'il faut pour le faire tourner
15. Ce que cette mémoire laisse passer
16. Ce que je retiens
17. À propos

Annexe A. Lexique

Annexe B. Revue avant de brancher une mémoire

Annexe C. La configuration, variable par variable

Annexe D. La ligne de commande

1. La mémoire qui se remplit toute seule

« Tu te rappelles ce que je t'ai dit sur ce projet ? »

La question arrive dès qu'un assistant sert à autre chose qu'une tâche isolée. Les applications y répondent par une mémoire automatique : le modèle repère dans la conversation ce qui lui semble durable, l'enregistre, le ressort au moment qu'il juge utile. Ce qui est enregistré est donc ce qu'il a cru comprendre. En relisant la mienne, j'y ai trouvé des décisions que j'avais changées depuis, des approximations recopiées d'un échange où je réfléchissais à voix haute, et des inférences que je n'avais jamais formulées.

Aucune de ces lignes n'est une invention. Elles sont le résidu d'un tri fait sans moi, sur un matériau qui contient autant d'hypothèses abandonnées que de conclusions. La difficulté ne tient pas à la qualité du tri. Elle tient à ce que personne ne me demande de valider ce qui entre, et que ce qui est entré revient ensuite avec le statut d'un fait.

loom-memory est la mémoire que j'ai écrite pour tenir l'inverse. J'y ajoute un texte, une page web ou un fichier markdown ; Claude interroge cette base avant de répondre sur mes projets, et il n'écrit dedans que si je le lui demande dans le message courant. Le contenu vit dans un dépôt, sur ma machine. Les deux modèles qui vectorisent et classent tournent en local. La seule sortie réseau du projet est le téléchargement d'une page, quand je demande de la mémoriser.

Le code tient en 1 957 lignes de Python, 561 lignes de tests et 40 cas de test. Ce cahier décrit ce que ce serveur garantit sur ce qui entre, par quel mécanisme, et où la garantie s'arrête.

Ce cahier est une étude de cas. Il décrit une implémentation, la mienne, plutôt qu'une méthode générale. Trois décisions s'en détachent et valent pour toute mémoire branchée sur un agent : une porte d'écriture tenue par une consigne dont on connaît la portée, un contrôle d'accès sur le seul tool qui lit le disque, et un réglage du rappel mesuré sur des cas de référence.

Trois consommateurs, une seule base

Claude Desktop lance un serveur MCP par surface : une instance pour le chat, une autre pour les sessions Cowork et Code. Mes agents LOOM appellent les mêmes tools par leur propre client. Cela fait trois consommateurs pour une base unique, et cette contrainte décide de choses qu'on n'attendrait pas à ce niveau, jusqu'au mode de stockage retenu. La partie 13 y revient.

2. Ce que « choisir ce qui entre » veut dire

La formule recouvre trois questions qu'on traite souvent comme une seule.

- **La porte.** Qui décide qu'un contenu entre : un modèle qui l'estime utile, ou moi qui le demande.
- **La source.** D'où vient le texte ajouté : ma frappe, une page que j'ai choisie, un fichier du disque.
- **La frontière.** Ce que le tool d'ajout atteint réellement sur la machine quand on lui passe un chemin.

Une mémoire automatique répond à la première en confiant la décision au modèle. Les deux autres ne se posent pas chez elle, puisque son matériau est la conversation en cours. Elles apparaissent dès qu'un tool sait ouvrir un fichier : `add_file` suivi de `get` devient alors une lecture de fichier arbitraire présentée comme une mémorisation.

Le partage compte parce que les deux familles n'ont pas le même profil d'erreur. Une lecture ratée se voit tout de suite : le modèle répond à côté, je relance la question autrement. Une écriture ratée s'installe. Elle sera relue dans trois semaines, dans une

conversation qui porte sur autre chose, par un modèle qui la présentera comme acquise parce qu'elle vient de ma propre mémoire.

Une écriture est une décision

Un agent qui écrit dans sa propre mémoire modifie ce qu'il lira au tour suivant, et ce que lira la conversation de la semaine prochaine. L'effet est différé, il ne s'annonce pas, et personne ne relit une base qu'il croit juste. C'est la raison pour laquelle je traite l'écriture comme une opération qu'on demande, et la lecture comme une opération qu'on encourage : la première engage l'avenir, la seconde coûte quelques secondes de recherche.

La suite de ce cahier décrit les quatre mécanismes qui appliquent ce partage : la séparation des tools en deux familles, la consigne portée par le serveur et par chaque description, le compte rendu que rend tout tool d'écriture, et le confinement du seul tool qui touche au disque.

3. Neuf tools, deux familles

Le serveur expose neuf tools. Quatre lisent, cinq écrivent, et la frontière entre les deux porte tout le reste du document.

Tool	Ce qu'il fait	Famille
search	Recherche hybride puis reclassement. Sans project, la recherche est globale et chaque résultat indique le sien. k est plafonné à 10.	Lecture
get	Texte intégral d'un document et ses métadonnées.	Lecture
list_docs	Derniers documents ajoutés, du plus récent au plus ancien, sans le texte.	Lecture
projects	Projets présents et nombre de documents pour chacun.	Lecture
add_text	Ajoute un texte brut, avec un titre et un projet.	Écriture
add_url	Télécharge une page, en extrait le contenu, l'ajoute sous le titre de la page.	Écriture
add_file	Ajoute un fichier markdown local, découpé par titres.	Écriture
update	Remplace le texte d'un document. Identifiant, titre, projet et tags conservés.	Écriture
delete	Supprime définitivement un document et ses morceaux.	Écriture

Les quatre premiers sont déclarés `read_only_hint` au sens MCP. L'annotation est une information donnée au client : elle lui permet de présenter ces appels sans demander confirmation, et de traiter les autres différemment. `update` porte `idempotent_hint`, `delete` porte `destructive_hint`.

Ces annotations ne bloquent rien. Elles décrivent l'intention de l'auteur du serveur à un client qui en fait ce qu'il veut. Un client qui les ignore appelle `delete` aussi facilement que `search`, et le serveur exécute.

Deux plafonds sont posés dans la signature des tools plutôt que dans le code : `k` est borné entre 1 et 10 pour `search`, `n` entre 1 et 50 pour `list_docs`. Un modèle qui demande cinquante résultats de recherche ne les lira pas mieux que cinq ; il remplira son contexte de passages de score médiocre, et la réponse s'en ressentira. La borne est déclarée dans le schéma, donc le client la voit et le modèle la respecte sans avoir à essayer.

4. La consigne d'écriture n'est pas un verrou

La règle du serveur tient en une phrase : les tools d'écriture ne sont appelés que sur demande explicite, jamais de la propre initiative du modèle. Elle apparaît à deux endroits, et c'est délibéré.

Le premier est le champ instructions du serveur, que le client place dans le contexte de la session.

```
Écriture : n'appelle JAMAIS `add_text`, `add_url`, `add_file`, `update` ou
`delete` de ta propre initiative. Uniquement quand Denis le demande
explicitement dans son message courant ("ajoute ça à ma mémoire",
"mémorise cette page", "supprime ce document"). En cas de doute,
demande-lui. Chaque tool d'écriture renvoie ce qu'il a réellement fait :
montre-le à Denis.
```

Le second est la description de chaque tool d'écriture, où la même phrase est concaténée.

```
_WRITE_GUARD = (
    " N'appeler QUE si Denis le demande explicitement dans son message "
    "courant ; jamais de ta propre initiative."
)

@mcp.tool(
    description="Supprime définitivement un document et ses chunks. Renvoie "
    "le résumé de ce qui a été supprimé." + _WRITE_GUARD,
    annotations={"destructive_hint": True},
)
async def delete(doc_id: str) -> DocSummary:
    return await _call(svc().delete(doc_id))
```

La répétition tient à une différence de portée. Les instructions du serveur sont lues une fois, au début de la session, et elles reculent dans le contexte à mesure que la conversation s'allonge. La description du tool, elle, accompagne le catalogue à chaque appel au modèle. Sur un échange de trente tours, la seconde est toujours là quand la première a été noyée.

Ce que cette consigne vaut, et ce qu'elle ne vaut pas

C'est une instruction adressée à un modèle de langage. Elle n'empêche rien. Un modèle qui décide d'appeler `add_text` l'appelle, et le serveur écrit. Je la tiens pour ce qu'elle est : un défaut de comportement, efficace parce que le cas normal est simple à reconnaître, inopérant dès que quelque chose pousse dans l'autre sens.

Ce qui la rendrait technique existe et coûte cher. Un serveur en lecture seule pour le client conversationnel, avec les écritures réservées à la ligne de commande, ferme la question et supprime l'usage que j'en fais le plus souvent, qui est de dire « mémorise cette page » en cours de discussion. Une confirmation hors bande, sur un canal que le modèle ne contrôle pas, serait la bonne réponse ; MCP n'offre aujourd'hui aucun mécanisme de ce genre, et le simuler par un second appel de tool revient à demander au modèle de se confirmer à lui-même.

Le compromis actuel déplace donc la garantie ailleurs : sur ce que l'écriture rend, et sur ce qu'elle peut atteindre.

5. Ce qu'un tool d'écriture rend

Chaque tool d'écriture renvoie un compte rendu de ce qu'il a fait, et la dernière phrase des instructions du serveur demande au modèle de le montrer.

Champ	Ce qu'il dit
<code>doc_id</code>	l'identifiant du document touché, celui que <code>get</code> et <code>delete</code> attendent
<code>title</code>	le titre retenu, qui vient de la page ou du premier titre du fichier
<code>project</code>	le projet de rattachement, normalisé en minuscules
<code>chunks</code>	le nombre de morceaux vectorisés ; 0 signale qu'aucune écriture n'a eu lieu
<code>updated</code>	vrai si un document existant a été remplacé plutôt que créé
<code>duplicate_of</code>	l'identifiant d'un document au contenu identique ; rien n'a été écrit

Un tool d'écriture qui rend « ok » laisse trois questions ouvertes : sur quel document, sous quel titre, et combien de fois. Les trois se paient plus tard, quand on cherche pourquoi la même note apparaît deux fois sous deux titres différents. Le compte rendu est aussi ce qui permet de rattraper une écriture non voulue tant que la conversation est sous les yeux : `doc_id` est là, `delete` prend `doc_id`.

`chunks` à zéro accompagné de `duplicate_of` est le cas le plus fréquent, et le plus utile à afficher. Il dit que le contenu était déjà là, sous un identifiant que je peux aller lire.

6. Le seul contrôle technique

Un serveur MCP tourne avec les droits de l'utilisateur qui a lancé le client. Claude Desktop ne lui applique aucune restriction de dossier. Sans garde-fou, `add_file` suivi de `get` lit n'importe quel fichier de la machine, y compris depuis une session Cowork dont les dossiers autorisés sont bien plus étroits. Ce n'est pas une hypothèse : c'est le test qui a motivé cette partie.

Trois contrôles s'appliquent, dans cet ordre, et le message d'erreur nomme toujours celui qui a mordu.

```
def resolve_allowed(path: str | Path, settings: Settings) -> Path:
    if not settings.allowed_roots:
        raise PathDeniedError(
            "add_file désactivé : aucune racine dans LOOM_MEMORY_ALLOWED_ROOTS")
    target = Path(path).expanduser().resolve()
    root = next((r for r in _resolved_roots(settings)
                 if target.is_relative_to(r)), None)
    if root is None:
        raise PathDeniedError(...)
    deny = (*BASE_DENY, *settings.deny_patterns)
    for part in target.relative_to(root).parts:
        for pattern in deny:
            if fnmatch.fnmatchcase(part, pattern):
                raise PathDeniedError(
                    f"chemin refusé par le motif '{pattern}' : {target}")
    if target.suffix.lower() not in ALLOWED_SUFFIXES:
        raise PathDeniedError(...)
    return target
```

- **Le confinement.** Le chemin est résolu, liens symboliques compris, puis comparé aux racines de `LOOM_MEMORY_ALLOWED_ROOTS`, elles-mêmes résolues. Un lien qui sort d'une racine est refusé. Sans racine configurée, `add_file` est refusé d'office : la valeur par défaut de ce réglage est la liste vide, donc l'installation par défaut ne lit aucun fichier.
- **Les motifs.** Aucun composant du chemin relatif à la racine ne doit correspondre à un motif refusé. Le contrôle porte sur chaque composant, donc `dev/projet/.venv/lib/note.md` est écarté par le motif `.venv` alors qu'aucun nom de fichier de ce chemin ne lui ressemble.
- **L'extension.** Trois valeurs acceptées : `.md`, `.markdown` et `.txt`. Une mémoire documentaire n'a pas de raison d'ouvrir un binaire, et le format d'entrée du découpage est le markdown.

La liste de base compte dix-neuf motifs, répartis en quatre familles : les fichiers d'environnement, les clés privées et certificats, les répertoires de configuration personnelle où dorment des identifiants, et les répertoires de machine sans intérêt documentaire.

```
BASE_DENY: tuple[str, ...] = (  
    ".env", ".env.*", "*.pem", "*.key", "*.p12", "*.pfx",  
    "id_rsa*", "id_ecdsa*", "id_ed25519*",  
    ".ssh", ".aws", ".gnupg", ".netrc", ".npmrc", ".pypirc",  
    ".git", ".venv", "node_modules", "__pycache__",  
)
```

LOOM_MEMORY_DENY_PATTERNS ajoute à cette liste et ne peut rien lui retirer. Une garantie qu'un fichier de configuration désactive n'en est plus une, et c'est le genre de ligne qu'on commente un soir pour un essai.

Le confinement est couvert par huit cas de test, plus du tiers de ce qui porte sur l'ingestion. Ils vérifient le refus sans racine configurée, le chemin hors racine, le lien symbolique qui sort d'une racine, un motif qui mord sur un répertoire intermédiaire, un motif qui mord sur le nom de fichier, une extension refusée, un motif ajouté par configuration, et le cas passant. Une fonction d'autorisation dont on ne teste que le cas passant ne prouve rien : elle prouve qu'elle laisse passer.

Ce que le filtrage par nom laisse passer

Le filtrage porte sur des noms. Un jeton d'API collé dans une note markdown d'une racine autorisée sera lu, indexé, et ressortira dans un extrait de recherche. Le mécanisme attrape des conventions, et une convention est une heuristique. J'ai traité ce même sujet plus en détail dans le cahier 04, sur un serveur de fichiers : les conclusions valent ici sans changement.

La frontière réelle est le choix des racines. Sur ma machine, `add_file` voit un seul répertoire, celui de mes dépôts. Les deux autres tools d'écriture ne posent pas la question : `add_url` sort sur le réseau sans toucher au disque, `add_text` ne lit rien du tout.

7. Le trajet d'une note

Les trois entrées convergent vers le même pipeline. Ce qui les distingue tient au seul travail d'extraction, avant que le texte ne devienne un document.

Le texte brut est pris tel quel : je fournis le titre et le projet, rien n'est deviné. Un fichier markdown est lu tel quel lui aussi, et son titre est le premier titre de niveau 1, à défaut le nom du fichier. Une page web demande davantage.

Une page web devient du markdown structuré

L'extraction retire d'abord le bruit évident par une expression XPath : navigation, pied de page, scripts, feuilles de style, cadres, formulaires, gabarits, et tout ce qui porte aria-hidden ou un rôle de navigation. Elle cherche ensuite un élément main, à défaut un article, à défaut le corps entier, puis convertit ce fragment en markdown avec les titres, les listes, les tableaux et les blocs de code conservés. Les images et les liens sont retirés de la conversion.

Le parti pris est de privilégier la structure plutôt que la détection fine du contenu principal. Les pages que je mémorise sont choisies une par une : un menu résiduel dans le texte coûte quelques mots, un titre perdu coûte un niveau entier dans le chemin de titres, et la partie 8 explique pourquoi ce chemin décide de la qualité du rappel.

Une correction occupe une fonction entière, et elle n'est pas là par excès de soin. Le HTML réel colle les éléments en ligne sans espace : un span de libellé suivi directement d'un texte produit « Ce qui arriveL'agent » après conversion. La fonction insère une espace quand un élément en ligne se termine par un caractère de mot et que la suite commence par une majuscule, ou quand il se termine par une ponctuation forte suivie d'une lettre. Les mots soudés survivent à la vectorisation et ne ressortent jamais dans une recherche.



8. Ce que le morceau emporte avec lui

Le texte est découpé en suivant les titres markdown. Chaque section devient un morceau, une section trop longue est retaillée en paragraphes, et un paragraphe encore trop long est coupé aux fins de phrase. Le découpage garde une pile de titres pendant le parcours, et sait ignorer les lignes de titre qui se trouvent à l'intérieur d'un bloc de code délimité.

Variable	Défaut	Rôle
CHUNK_TARGET_CHARS	1800	taille visée d'un morceau
CHUNK_MAX_CHARS	2400	au-delà, une section est retaillée
CHUNK_OVERLAP_CHARS	180	chevauchement entre morceaux consécutifs d'une même section
CHUNK_MIN_CHARS	200	en dessous, un morceau est fusionné avec le précédent

Ces quatre valeurs sont en caractères, avec l'approximation de quatre caractères par jeton en français. C'est un raccourci assumé : le vrai découpage en jetons dépend du modèle, et

le mesurer à chaque appel coûterait un passage de tokenizer pour un réglage dont la sensibilité est faible.

Le point qui compte se joue juste avant la vectorisation. Chaque morceau est préfixé par son chemin de titres.

```
class Chunk(BaseModel):
    doc_id: str
    index: int
    heading_path: str
    text: str

    @property
    def embed_text(self) -> str:
        """Texte réellement vectorisé : le chemin de titres préfixe le contenu."""
        return f"{self.heading_path}\n\n{self.text}" if self.heading_path else self.text
```

Le chemin ressemble à « loom-fs, serveur MCP filesystem > Sécurité > Motifs de refus ». Un paragraphe isolé de son titre perd ce dont il parle : il dit « la liste est dans le code » sans dire de quelle liste ni de quel projet. La question posée à la recherche, elle, contient toujours le sujet. De tous les réglages de ce serveur, c'est celui qui a le plus déplacé le rappel, et il tient en une ligne.

La retaille d'une section trop longue mérite un mot, parce que ses deux détails se paient à la lecture. Le chevauchement est repris à la fin du morceau précédent et coupé au premier espace rencontré, donc il commence sur un mot entier.

L'autre détail est la fusion des morceaux trop courts. Un titre suivi de deux lignes produit un morceau de cent caractères, qui ne porte pas assez de matière pour être retrouvé et occupe une place dans les résultats. Il est absorbé par le précédent quand le total reste sous le maximum. Le chemin de titres conservé est alors celui du morceau d'accueil, ce qui est un compromis assumé : la sous-section fusionnée perd son propre titre dans le préfixe, et gagne d'exister.

Les lignes de titre situées à l'intérieur d'un bloc de code délimité sont ignorées par le découpage. Sans cette précaution, un commentaire Python commençant par un dièse ouvrirait une section, et le bloc de code serait coupé en deux au milieu.

Le texte rendu par search reste le morceau seul, sans son préfixe. Le chemin voyage à part, dans le champ heading_path, où il sert à situer l'extrait pour le lecteur.

9. Deux fois le même texte

Une mémoire alimentée à la main reçoit deux fois le même contenu plus souvent qu'on ne l'imagine : la page qu'on mémorise une seconde fois six mois plus tard, le fichier README qu'on réajoute après l'avoir modifié, la note recopiée depuis une autre conversation.

Une empreinte SHA-256 du texte normalisé tranche le cas. La normalisation applique NFKC, replie la casse et écrase les suites d'espaces, donc un copier-coller reformaté produit la même empreinte que l'original.

```
def normalize(text: str) -> str:
    text = unicodedata.normalize("NFKC", text).casefold()
    return _WS.sub(" ", text).strip()

def content_hash(text: str) -> str:
    return hashlib.sha256(normalize(text).encode("utf-8")).hexdigest()
```

Un contenu déjà présent n'est pas réécrit : le tool rend `duplicate_of` et zéro morceau. Une source déjà connue, URL ou chemin de fichier, dont le contenu a changé remplace le document existant au lieu d'en créer un second. Le document garde son identifiant, sa date d'ajout d'origine, et reçoit une date de mise à jour.

Réécrire sans laisser d'orphelin

Les identifiants de morceau sont dérivés de l'identifiant du document et de l'index du morceau, par un UUID de version 5 sur un espace de noms fixe.

```
_NAMESPACE = uuid.UUID("6f1c3a2e-9b7d-4e5a-8c0f-2d1e4b6a7c93")

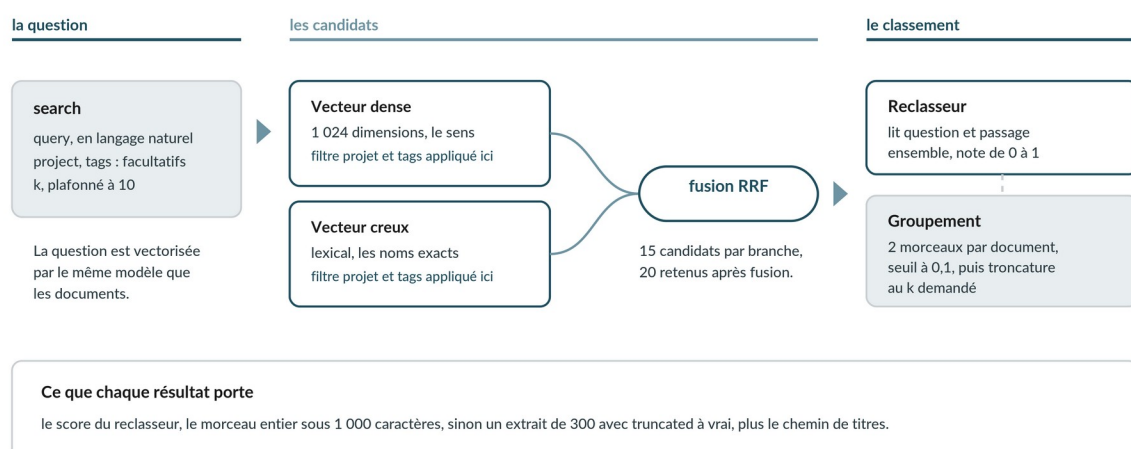
def point_id(doc_id: str, chunk_index: int) -> str:
    return str(uuid.uuid5(_NAMESPACE, f"{doc_id}:{chunk_index}"))
```

Une réécriture commence par supprimer les morceaux du document, puis réécrit les nouveaux aux mêmes positions. Un document qui passe de douze à neuf morceaux ne laisse donc rien derrière lui. Sans cette propriété, les trois morceaux en trop resteraient indexés, sortiraient dans les recherches, et rendraient un texte que get ne montre plus. Une mémoire qui contredit son propre document est pire qu'une mémoire vide.

10. Retrouver : hybride, puis reclassement

La recherche se déroule en trois temps, et chacun corrige une faiblesse du précédent.

BGE-M3 produit en un seul passage un vecteur dense de 1 024 dimensions et un vecteur creux lexical. Qdrant lance deux recherches sur ces deux représentations, puis fusionne les listes par RRF. Le dense rattrape les reformulations, le creux rattrape les noms propres et les identifiants exacts, qu'une représentation dense dilue.



Les filtres de projet et de tags sont posés dans chacune des deux branches, avant la fusion.

```
prefetch=[
  qm.Prefetch(query=query.dense, using="dense",
    limit=self._s.prefetch_limit, filter=flt),
  qm.Prefetch(query=qm.SparseVector(indices=query.sparse.indices,
    values=query.sparse.values),
    using="sparse", limit=self._s.prefetch_limit, filter=flt),
],
query=qm.FusionQuery(fusion=qm.Fusion.RRF),
```

Filtrer après la fusion donnerait un nombre de résultats qui varie avec le projet demandé : sur quinze candidats par branche, un projet peu représenté en fournirait deux, et la recherche rendrait deux résultats là où elle en promet cinq. Filtrer dans la branche fait que les quinze candidats appartiennent tous au périmètre demandé.

Les candidats passent ensuite dans un cross-encoder, bge-reranker-v2-m3, qui lit la question et le passage ensemble et rend une note entre 0 et 1. C'est le poste de coût dominant, et la raison pour laquelle il en reçoit vingt et pas deux cents. Les résultats sont enfin regroupés à deux morceaux par document au maximum, filtrés par le seuil, puis tronqués au k demandé.

Le nombre de candidats est le seul réglage de cette chaîne qui se paie en secondes. Le reclasseur lit vingt couples question-passage par recherche, par lots de seize, et c'est lui qui fixe le temps de réponse. Doubler ce nombre double le temps pour un gain que la mesure des cas de référence n'a pas montré : au-delà du vingtième candidat, la fusion RRF ne remonte plus grand-chose qui mérite d'être relu.

Le groupement par document

Sans plafond par document, une note longue et bien écrite occupe les cinq places : ses sept morceaux parlent tous du sujet, et le reste de la mémoire disparaît. Deux morceaux laissent de la place à un second document tout en gardant assez de contexte quand la réponse est à cheval sur deux sections voisines.

11. Ce que le score veut dire

Le score rendu est celui du reclasseur, normalisé entre 0 et 1. Il se lit, et cette lecture a demandé une centaine de recherches avant de se stabiliser.

Score	Ce que le passage contient	Ce que j'en fais
au-dessus de 0,8	la réponse à la question posée	je lis l'extrait et je m'arrête là
de 0,3 à 0,8	le sujet, sans y répondre franchement	j'appelle get pour lire le document entier
de 0,1 à 0,3	un voisinage thématique	utile en recherche globale, rarement autrement

Score	Ce que le passage contient	Ce que j'en fais
en dessous de 0,1	du bruit ramené faute de mieux	écarté par le seuil

La longueur de l'extrait suit une règle simple : en dessous de 1 000 caractères, search rend le morceau entier. Cela couvre la majorité de mes notes et évite de couper un tableau au milieu, ce qui est la manière la plus efficace de rendre une donnée fausse. Au-delà, l'extrait fait 300 caractères, coupés au dernier espace, et le champ truncated passe à vrai. Ce champ est le signal qui dit au modèle d'appeler get.

12. Régler le seuil sur des cas de référence

Le seuil par défaut est à 0,1. Ce chiffre vient d'une mesure, et la mesure se rejoue à chaque changement de découpage, de modèle ou de fusion.

Le fichier data/golden.jsonl porte un cas par ligne : la question telle que je la poserais, l'identifiant du document qui doit sortir, et deux champs facultatifs, title pour relire le fichier et project quand la question serait posée dans ce périmètre.

```
{"query": "comment purger le journal d'audit sans arrêter le serveur",
"doc_id": "f3c9d186-c844-427b-89c5-0ed64f515c88"}
```

La commande loom-memory eval rejoue chaque question à seuil zéro, donc sans rien écarter, et rend par cas le rang du document attendu et son score. Elle calcule ensuite trois métriques usuelles, rappel au premier rang, rappel dans les cinq premiers et rang réciproque moyen, puis deux valeurs qui servent directement au réglage.

- **Le seuil suggéré** est le plus bas score obtenu par un document attendu. Le placer là ne fait perdre aucun cas actuellement trouvé.
- **Le bruit écarté à ce seuil** est la part des résultats hors document attendu qui tomberaient en dessous. C'est ce que le seuil achète.

Le jeu s'enrichit à chaque document ajouté, à raison de deux questions, avec eval-add. Il compte aujourd'hui huit cas sur deux documents, ce qui est peu pour une métrique et déjà suffisant pour attraper une régression de découpage.

La mesure se lance à seuil zéro délibérément. Un cas dont le document attendu sort au rang sept avec un score de 0,07 est une information utile : il dit que le rappel fonctionne et que le seuil coupe trop haut. Le même cas mesuré au seuil courant serait compté comme un échec complet, et on chercherait le défaut dans le découpage.

Un mauvais score signale souvent un mauvais document

Le résultat le plus utile de cette mesure n'est pas le réglage du seuil. C'est qu'un cas qui sort avec un score faible alors que le document contient la réponse pointe presque

toujours vers un document mal rédigé. Le cas type est une commande posée sans la phrase qui dit quand et pourquoi l'utiliser : le morceau contient la réponse et aucun des mots de la question.

Sur mes premiers cas, la même question passait de 0,91 à 0,11 selon que le document gardait ou non ses deux phrases de contexte. Un facteur huit sur le score, pour deux phrases qu'un rédacteur pressé coupe en se disant que la commande parle d'elle-même.

Depuis, j'écris les notes de cette mémoire en sachant qu'elles seront découpées : un titre par sujet, une phrase de contexte avant chaque bloc technique, et le nom du projet dans le titre du document. Ce sont les mêmes règles qu'un bon README, rendues mesurables.

13. Deux collections, une source de vérité

Le stockage tient en deux collections Qdrant, et la distinction entre les deux est la seule décision d'architecture qui compte ici.

- **documents** porte le texte intégral et les métadonnées, sans vecteur. C'est la source de vérité.
- **memory** porte les morceaux vectorisés, dense et creux. Elle se reconstruit entièrement depuis la première.

La commande `loom-memory` reindex supprime la collection de morceaux, la recrée, et réindexe chaque document avec le modèle courant. L'opération est sans risque parce que rien d'irremplaçable ne vit dans l'index : un index est un cache coûteux à reconstruire, jamais une donnée.

La sauvegarde de référence est un export JSONL, un document par ligne, texte et métadonnées compris. Ce fichier est commité avec le dépôt, donc indépendant de Qdrant comme du modèle d'embedding. Le répertoire de stockage de Qdrant, lui, est ignoré par git.

Le verrou de modèle

Le nom du modèle dense et sa dimension sont écrits dans `data/meta.json` au moment de l'indexation. Si la configuration en demande un autre, le serveur refuse de démarrer et dit quoi lancer.

```
raise ModelMismatchError(
    f"la base a été indexée avec {meta.get('dense_model')} "
    f"({meta.get('dense_dim')}), la configuration demande {dense_model} "
    f"({dense_dim}) : lance `loom-memory reindex`"
)
```

Deux modèles d'embedding dans le même index ne produisent aucune erreur visible. Les vecteurs ont la bonne dimension, la recherche s'exécute, et les résultats sont subtilement faux pour les documents indexés avec l'ancien. Une panne silencieuse dans un composant

dont la sortie est déjà approximative peut vivre des mois. Refuser de démarrer coûte trente secondes et supprime la classe entière.

Pourquoi Qdrant tourne dans un conteneur

Le mode embarqué de Qdrant ouvre son stockage en exclusif : un seul processus à la fois. Or Claude Desktop lance deux instances du serveur, une pour le chat et une pour les sessions Cowork et Code. La seconde mourrait à l'ouverture, sans message lisible dans l'application.

```
docker run -d --name qdrant --restart unless-stopped \
--user "$(id -u):$(id -g)" \
-e QDRANT__STORAGE__SNAPSHOTS_PATH=/qdrant/storage/snapshots \
-p 127.0.0.1:6333:6333 \
-v /home/denis/dev/loom-memory/data/qdrant:/qdrant/storage \
qdrant/qdrant
```

Trois détails de cette commande se paient si on les saute. L'option `--user` fait que les fichiers de `data/qdrant` appartiennent à l'utilisateur ; sans elle, le dépôt se remplit de fichiers appartenant à `root`. La variable de chemin des instantanés devient alors nécessaire, puisque Qdrant ne peut plus écrire son dossier d'instantanés dans l'image. Le port est lié à l'interface locale, parce qu'un Qdrant par défaut n'authentifie personne.

Le mode embarqué reste disponible en l'absence d'URL, et c'est celui qu'utilisent les tests. Les deux modes n'ont pas le même format sur disque : passer de l'un à l'autre demande un export puis un import.

14. Ce qu'il faut pour le faire tourner

Deux modèles de 570 millions de paramètres tournent sur la machine. Les poids de BGE-M3 pèsent environ 3 Go, ceux du reclasseur environ 2,3 Go ; ils sont téléchargés depuis Hugging Face au premier appel et mis en cache dans le répertoire personnel. Il n'y a rien d'autre à télécharger ensuite, et plus aucune sortie réseau en dehors de `add_url`.

Sur GPU, les deux tiennent dans 3 Go de mémoire vidéo en demi-précision. Sur processeur, avec `LOOM_MEMORY_DEVICE` à `cpu`, la demi-précision est coupée automatiquement et le reclasseur devient le poste dominant : comptez une dizaine de secondes par recherche. C'est utilisable pour une ligne de commande, pénible au milieu d'une conversation.

Le chargement des modèles est sorti du chemin de la première requête. Le serveur répond au handshake MCP tout de suite, puis lance une tâche de fond qui force le chargement en vectorisant un mot et en notant un couple factice.

```
async def warmup(self) -> None:
    """Charge les modèles hors du chemin de la première requête."""
    emb = await asyncio.to_thread(self._embedder.embed_query, "warmup")
    await asyncio.to_thread(self._retriever.reranker.score, "warmup", ["warmup"])
    del emb
```

Le détournement par un thread revient à chaque appel de modèle. FlagEmbedding est synchrone et occupe le processeur pendant plusieurs centaines de millisecondes ; l'exécuter directement dans la boucle asynchrone bloquerait le serveur MCP, qui ne pourrait plus répondre au client pendant ce temps.

Les pannes qui ne disent pas leur nom

Symptôme	Cause et correctif
Échec au démarrage, « Connection closed » dans le client	Qdrant injoignable, ou un environnement virtuel antérieur à l'ajout du script ; la première ligne d'erreur du serveur, dans les journaux de l'application, nomme lequel des deux
Refus de démarrer en parlant de modèle	data/meta.json porte un autre modèle que la configuration ; loom-memory reindex reconstruit l'index
add_file refusé	le message nomme la règle qui a mordu : racine, motif ou extension ; sans racine configurée, tout est refusé
Une recherche prend dix secondes	le reclasseur tourne sur processeur ; vérifier LOOM_MEMORY_DEVICE et que torch voit la carte
IndexError dans FlagEmbedding au premier appel	la carte est vue par torch mais sans noyaux compatibles ; FlagEmbedding attrape l'erreur CUDA, réduit son lot jusqu'à zéro, et plante sur une liste vide

Le dernier cas a coûté une soirée. Une carte de génération Pascal, capacité de calcul inférieure à 7.5, n'est pas prise en charge par les roues torch CUDA 13 publiées sur PyPI. Le message d'erreur parle d'une liste vide dans une bibliothèque tierce, et ne dit rien de la carte. Il faut alors le processeur, ou une roue construite pour une version de CUDA plus ancienne.

Une contrainte du même ordre est inscrite dans les dépendances, avec son motif à côté : FlagEmbedding 1.4 casse avec transformers 5, dont la méthode pad reçoit une liste là où elle attendait un dictionnaire. Une borne de version sans commentaire est une ligne que quelqu'un relèvera dans six mois en pensant bien faire.

15. Ce que cette mémoire laisse passer

Cette partie existe parce qu'un lecteur qui découvre ces limites après coup a raison de se demander ce qu'on lui a caché d'autre.

Une page mémorisée peut contenir des instructions

add_url télécharge une page choisie par moi, et le contenu de cette page entre ensuite dans le contexte de l'agent au premier résultat de recherche. Si elle porte des instructions déguisées en texte, elles arrivent dans la conversation comme le reste.

Ce qu'elles ne peuvent pas faire, c'est atteindre le disque : `add_file` reste confiné à ses racines, et aucun autre tool ne lit de fichier. Ce qu'elles pourraient faire, c'est pousser le modèle à écrire dans la mémoire, puisque la porte d'écriture est une consigne. Le compte rendu de chaque écriture est la contre-mesure, et il ne vaut que si je le lis.

Aucun quota, aucune limitation de débit

Un agent parti en boucle peut lancer autant de recherches qu'il veut. Chacune coûte un passage de reclasseur, donc du temps, sur ma carte graphique et sur mon électricité. Rien ne le borne aujourd'hui. La question deviendra sérieuse le jour où cette mémoire alimentera un agent que personne ne regarde ; en usage interactif, la personne qui suit la conversation est le plafond.

Deux tools parcourent toute la collection

`list_docs` et `projects` lisent l'ensemble de la collection documents pour trier par date ou compter par projet. Le coût est linéaire, invisible jusqu'à quelques milliers de documents, et il le restera longtemps pour une base alimentée à la main. Le jour où il faudra le traiter, la réponse est un index sur le champ de date plutôt qu'un parcours.

Le serveur tourne avec vos droits

Comme tout serveur MCP en stdio, c'est un processus lancé par l'application cliente, sans machine virtuelle ni utilisateur dédié. Tout ce qui précède est de la discipline interne à ce processus. Un défaut dans la fonction de résolution de chemin donnerait accès au disque entier, et c'est la raison pour laquelle elle est courte, avec un seul chemin de retour, et couverte par huit des quarante tests.

16. Ce que je retiens

La décision qui structure tout le reste tient en une phrase de configuration : la liste des racines lisibles est vide par défaut. Une mémoire installée sans réglage ne lit aucun fichier de la machine, et il faut un geste délibéré pour lui en ouvrir un. Les valeurs par défaut sont la vraie politique de sécurité d'un outil, puisque ce sont elles qui tournent chez les gens.

La deuxième est d'avoir accepté que la porte d'écriture soit une consigne, et de l'avoir écrit noir sur blanc plutôt que de laisser croire à un verrou. Une garantie annoncée qui n'en est pas coûte plus cher que l'absence de garantie : elle dispense de regarder.

La troisième est le chemin de titres préfixé au morceau avant vectorisation. Une ligne de code, un effet sur le rappel que rien d'autre n'a égalé, et une conséquence sur ma façon d'écrire : je rédige désormais mes notes en sachant à quoi ressemblera leur découpage.

Reste la question que ce cahier ne tranche pas. Une mémoire dont l'écriture serait confirmée hors de la conversation, sur un canal que le modèle ne peut pas fabriquer,

serait la bonne réponse au problème posé en partie 4. Le protocole ne l'offre pas aujourd'hui. En attendant, je lis les comptes rendus.

17. À propos

Je conçois et j'intègre des agents IA destinés à tourner en production, pour des PME et des artisans. Ce cahier décrit un outil de mon propre poste de travail, celui par lequel passent les agents avec lesquels j'écris le reste.

J'écris et je maintiens un socle d'orchestration public. Il porte les mécanismes décrits dans cette collection : validation des arguments avant appel, exécution parallèle bornée, checkpoints et reprise.

Le code de cette mémoire est public : github.com/denislamard/loom-memory

Le socle d'orchestration : github.com/denislamard/loom

Le détail technique et les cas d'usage : denislamard.github.io

Annexe A. Lexique

Terme	Ce qu'il désigne ici
document	une unité ajoutée en une fois, avec un titre, un projet et un texte intégral
morceau	une tranche de document, vectorisée et indexée ; l'unité que la recherche rend
chemin de titres	la suite des titres qui mènent à un morceau, préfixée au texte avant vectorisation
vecteur dense	la représentation numérique du sens d'un texte, 1 024 dimensions ici
vecteur creux	une représentation lexicale pondérée, qui garde les mots rares et les identifiants
recherche hybride	l'exécution des deux recherches puis la fusion de leurs classements
RRF	fusion par rang réciproque : un document bien classé dans les deux listes remonte
reclasseur	un modèle qui lit question et passage ensemble et note leur correspondance
cas de référence	une question dont on connaît le document qui doit sortir, gardée pour mesurer
seuil	le score en dessous duquel un résultat est écarté, réglé sur les cas de référence
racine autorisée	un répertoire sous lequel add_file accepte de lire, et rien en dehors

Annexe B. Revue avant de brancher une mémoire

Une page à parcourir avant de laisser un agent écrire dans une base qu'on relira. Elle vaut pour loom-memory comme pour n'importe quelle mémoire branchée par MCP.

La porte

- La liste des tools d'écriture est connue, et chacun a été appelé une fois à la main.
- Les tools de lecture portent l'annotation `read_only_hint`, les destructifs la leur.
- La consigne d'écriture figure dans les instructions du serveur et dans chaque description de tool d'écriture.

La frontière

- Le tool qui lit des fichiers refuse par défaut, tant qu'aucune racine n'est configurée.
- Les racines déclarées sont les répertoires les plus étroits qui permettent encore de travailler.
- Les motifs de refus s'appliquent à tous les composants d'un chemin, et la liste de base n'est pas désactivable par configuration.

Le contenu

- Chaque document porte un titre qui nomme son projet, et un titre par sujet à l'intérieur.
- Chaque bloc technique est précédé de la phrase qui dit quand et pourquoi l'utiliser.
- Les documents qui portent un secret en clair n'entrent pas, puisque le filtrage porte sur les noms de fichiers.

La mesure

- Deux questions de référence accompagnent chaque document ajouté.
- Le seuil vient d'une mesure, et la mesure est rejouée après tout changement de découpage, de modèle ou de fusion.
- Un changement de modèle d'embedding fait échouer le démarrage tant que l'index n'a pas été reconstruit.

Annexe C. La configuration, variable par variable

Toutes les options sont des variables d'environnement préfixées `LOOM_MEMORY_`, ou un fichier `.env` à la racine du dépôt, ignoré par git. Les valeurs ci-dessous sont les défauts.

Variable	Défaut	Rôle
DATA_DIR	data/	données : Qdrant embarqué, meta.json, export, cas de référence
QDRANT_URL	vide	URL d'un serveur Qdrant ; vide, le mode embarqué s'applique
QDRANT_API_KEY	vide	clé d'API si le serveur en exige une
DEVICE	cuda	périphérique torch ; cpu coupe le fp16 automatiquement
DENSE_MODEL	BAAI/bge-m3	modèle d'embedding ; en changer impose un reindex
RERANKER_MODEL	BAAI/bge-reranker-v2-m3	le cross-encoder de reclassement
ALLOWED_ROOTS	vide	racines lisibles par add_file ; vide, add_file est refusé
DENY_PATTERNS	vide	motifs refusés en plus de la liste de base, jamais à la place
PREFETCH_LIMIT	15	candidats par branche, dense et creuse, avant fusion
RERANK_CANDIDATES	20	candidats passés au reclasseur
MAX_CHUNKS_PER_DOC	2	morceaux d'un même document dans les résultats
FULL_CHUNK_CHARS	1000	en dessous, search rend le morceau entier
SNIPPET_CHARS	300	longueur de l'extrait au-delà
MIN_SCORE	0.1	score minimal d'un résultat, réglé par loom-memory eval
WARMUP_ON_START	true	préchargement des modèles en tâche de fond au démarrage
FAKE_MODELS	false	modèles factices, pour les tests ou une démonstration sans GPU
FETCH_TIMEOUT_S	20	délai de téléchargement pour add_url

Le branchement dans Claude Desktop pointe le binaire du venv plutôt que uv run. Entre l'exécution et le handshake MCP, uv peut résoudre des dépendances, voire télécharger un interpréteur, sans rien écrire sur la sortie standard pendant ce temps ; le client, lui, attend et conclut à un échec.

```
"loom-memory": {
  "command": "/home/denis/dev/loom-memory/.venv/bin/loom-memory-mcp",
  "env": {
    "LOOM_MEMORY_QDRANT_URL": "http://127.0.0.1:6333",
    "LOOM_MEMORY_ALLOWED_ROOTS": "/home/denis/dev",
    "LOOM_MEMORY_DEVICE": "cpu"
  }
}
```

Le serveur répond au handshake immédiatement et charge les modèles en tâche de fond. La première recherche peut attendre la fin de ce chargement ; les suivantes non.

Annexe D. La ligne de commande

loom-memory expose les mêmes opérations que le serveur, plus la maintenance. En mode serveur Qdrant, elle fonctionne pendant que le client tourne, ce qui permet d'ajouter et de vérifier sans fermer l'application.

Commande	Ce qu'elle fait
add-text PROJET TITRE [TEXTE]	ajoute un texte brut ; lu sur l'entrée standard si absent
add-url PROJET URL	télécharge une page et l'ajoute sous son titre
add-file PROJET CHEMIN	ajoute un fichier markdown, soumis aux racines autorisées
search "question" [-p] [-t] [-k]	recherche, avec filtre de projet, de tags et nombre de résultats
get DOC_ID	texte intégral d'un document
list [-p projet] [-n 20]	derniers documents ajoutés
projects	projets présents et nombre de documents
delete DOC_ID	supprime un document et ses morceaux
export [FICHIER]	sauvegarde JSONL, par défaut data/export.jsonl
import [FICHIER]	réimport ; les identifiants déjà présents sont ignorés
reindex	reconstruit les morceaux depuis les documents, avec le modèle courant
eval [FICHIER] [--json]	rejoue les cas de référence et propose un seuil
eval-add "question" DOC_ID	ajoute un cas de référence au jeu

Les options globales --fake, qui remplace les deux modèles par des factices déterministes, et --data-dir se placent avant la commande. La première est aussi ce qui fait tourner les quarante tests sans GPU et sans téléchargement.